

WebApp - Visão Geral da Arquitetura

Resumo integrado: aplicação, infraestrutura e atualização de dados

1. Objetivo

Este documento reúne, em uma única visão, os três pilares já documentados no projeto WebApp: arquitetura MVC, infraestrutura de produção e processo AtualizaIndicador. A proposta é servir como apresentação rápida do sistema e mostrar, em ordem lógica, como cada parte se conecta.

2. Visão completa

```

FONTES  EXTERNAS  USUÁRIO
|  |
v  |
+-----+-----+ |
| AtualizaIndicador | |
| Python  |  |
+-----+-----+ |
|  |
| grava  | HTTPS
v  v
+-----+ consulta +-----+
| PostgreSQL | <-----+-----+ | Flask |
+-----+ +-----+-----+
^
|
Gunicorn
^
|
Apache
^
|
UFW
^
|
INTERNET
```

3. Etapa 1 - Organização interna: MVC

A primeira camada conceitual é a organização do próprio WebApp. O projeto utiliza a separação de responsabilidades do padrão MVC.

MVC	WebApp	Responsabilidade
Model	models/ + database/	Entidades, dados, persistência e acesso ao PostgreSQL.
View	templates/ + static/	Interface apresentada ao usuário.
Controller	routes/	Recebe requisições e coordena o fluxo da aplicação.
Bootstrap	app.py	Inicializa o Flask e registra os componentes da aplicação.

```

Navegador
|
routes/ -> Controller
|
models/ -> Model
|
database/
|
templates/ -> View
|
Navegador
```

4. Etapa 2 - Preparação para produção

Depois da organização interna do código, a aplicação precisa ser executada de forma adequada em produção. O Flask não fica diretamente exposto à Internet. O Gunicorn executa a aplicação WSGI e escuta apenas na interface local da VPS.

```
systemd
|
+-- webapp.service
|
+-- uv run gunicorn
|
+-- 127.0.0.1:5153
|
+-- Flask app:app
```

O **uv** mantém o ambiente Python e suas dependências; o **systemd** mantém o serviço ativo; e o **Gunicorn** serve a aplicação Flask em produção.

5. Etapa 3 - Exposição do WebApp na Internet

A entrada pública é feita pelo Apache. O navegador acessa o domínio, o firewall UFW controla as portas disponíveis e o Apache recebe HTTP/HTTPS, atuando como reverse proxy para o Gunicorn.

```
Usuário
|
| https://webapp.jhan.com.br
v
Internet
|
v
UFW :443
|
v
Apache
|
| Reverse Proxy
v
127.0.0.1:5153
|
v
Gunicorn
|
v
Flask
```

6. Etapa 4 - HTTPS e certificado

A criptografia HTTPS termina no Apache. O certificado TLS protege o trecho público entre o navegador e o servidor. Internamente, o Apache encaminha a requisição ao Gunicorn em 127.0.0.1:5153.

```
EXTERNO
Navegador ===== HTTPS/TLS =====> Apache :443

INTERNO
Apache ----- HTTP -----> Gunicorn :5153
|
v
Flask
```

O certificado pode ser emitido e renovado por uma autoridade como Let's Encrypt, normalmente com Certbot. O Flask não precisa gerenciar diretamente o certificado.

7. Etapa 5 - Persistência no PostgreSQL

O PostgreSQL é o ponto central de persistência. O WebApp consulta e altera dados por meio da aplicação Flask. O navegador nunca acessa diretamente o banco.

```
Usuário -> Apache -> Gunicorn -> Flask -> PostgreSQL
<-
resposta
```

8. Etapa 6 - AtualizaIndicador

A atualização dos indicadores foi separada da aplicação web. O AtualizaIndicador é um processo Python independente que consulta fontes externas, trata os dados e os grava no PostgreSQL.

```
APIs externas
|
v
Scheduler
|
v
Coleta
|
v
Tratamento
|
v
Validação
|
v
PostgreSQL
|
v
WebApp
```

Essa decisão cria uma divisão simples: **AtualizaIndicador produz e atualiza os dados; PostgreSQL armazena; WebApp consulta e apresenta.**

9. O que depende de quê?

Componente	Depende principalmente de	Não precisa depender de
WebApp / Flask	Gunicorn, ambiente Python e PostgreSQL.	APIs de indicadores em tempo real.
Gunicorn	Flask e ambiente Python.	Apache para funcionar localmente.
Apache	Rede, domínio e backend local disponível.	Regras internas do Flask.
AtualizaIndicador	APIs externas, Python e PostgreSQL.	Apache, Gunicorn e rotas Flask.
PostgreSQL	Servidor e configuração do banco.	Interface web para armazenar dados.

10. Fluxo cronológico de uma requisição do usuário

1. Usuário acessa <https://webapp.jhan.com.br>
2. DNS direciona o domínio para a VPS
3. UFW permite a conexão na porta 443
4. Apache recebe a conexão HTTPS
5. TLS protege e valida a comunicação
6. Apache encaminha para 127.0.0.1:5153
7. Gunicorn recebe a requisição
8. Gunicorn entrega a requisição ao Flask
9. routes/ executa o Controller
10. models/database acessam o PostgreSQL
11. templates/static formam a View
12. A resposta retorna ao navegador

11. Fluxo cronológico de atualização dos indicadores

1. Scheduler identifica o horário de execução
2. AtualizaIndicador inicia a rotina
3. API externa é consultada
4. Dados brutos são recebidos
5. Python trata e padroniza os dados
6. Valores são validados
7. Dados são gravados no PostgreSQL
8. WebApp passa a consultar o dado atualizado
9. Usuário visualiza o indicador na interface

12. Diagnóstico geral

Em caso de problema, primeiro identifique em qual fluxo ocorreu a falha: acesso ao site ou atualização de dados.

```
SITE INDISPONÍVEL
DNS -> UFW -> Apache -> TLS -> Proxy -> systemd
-> Gunicorn -> Flask -> PostgreSQL

INDICADOR DESATUALIZADO
Scheduler -> API -> coleta -> tratamento -> validação
-> PostgreSQL -> consulta do WebApp
```

13. Resumo de cada tecnologia

Tecnologia	Resumo mental
MVC	Organiza as responsabilidades internas do WebApp.
Flask	Executa a lógica da aplicação web.
Gunicorn	Serve a aplicação Flask em produção.
uv	Gerencia o ambiente e as dependências Python.
systemd	Mantém o WebApp funcionando como serviço.
Apache	Recebe HTTP/HTTPS e atua como reverse proxy.
TLS / HTTPS	Protege a comunicação externa.
UFW	Controla as portas expostas pela VPS.
PostgreSQL	Centraliza a persistência dos dados.
AtualizaIndicador	Coleta, trata e atualiza indicadores independentemente do WebApp.

14. Mapa mental final

WEBAPP - VISÃO GERAL

```
+-----+
| USUÁRIO |
+-----+
|
| HTTPS
|
+-----v-----+
| UFW / APACHE |
| TLS + Proxy |
+-----+-----+
|
+-----v-----+
| GUNICORN |
+-----+-----+
```

```

|
+-----v-----+
| FLASK |
| MVC WebApp |
+-----+-----+
|
v
+-----+-----+
| POSTGRESQL |
+-----^-----+
|
+-----+-----+
| AtualizaIndic. |
| Python/Scheduler|
+-----^-----+
|
APIs externas

```

15. Conclusão

O WebApp é composto por duas linhas principais que se encontram no PostgreSQL. A primeira é a aplicação web: MVC organiza o código, Gunicorn executa o Flask, Apache publica o serviço e HTTPS protege o acesso. A segunda é o AtualizaIndicador: um processo Python independente que coleta e prepara dados externos. Essa separação deixa o sistema mais organizado, desacoplado e simples de manter.

Em resumo: **o AtualizaIndicador mantém os dados prontos; o PostgreSQL centraliza a persistência; o Flask aplica as regras e monta a resposta; Gunicorn executa o Flask; Apache publica o serviço; e HTTPS protege a comunicação com o usuário.**